

6.7960 DEEP LEARNING

Neural Network: function approximator

$x \rightarrow$ multiple layers $\rightarrow y = f(x; \theta)$

each layer

learnable parameters (weights w & biases b)

non-linear activation (ex. ReLU, tanh)

$x \rightarrow h_1 = \sigma(W_1 x + b_1) \rightarrow h_2 = \sigma(W_2 h_1 + b_2) \rightarrow \dots \rightarrow \hat{y}$

Linear Layers: (fully-connected layer)

$y = Wx + b$

$W \in \mathbb{R}^{d_{out} \times d_{in}}$

$b \in \mathbb{R}^{d_{out}}$

Non-linear activation functions:

ReLU $f(x) = \max(0, x)$

Derivative: 1 if $x > 0$, else 0

tanh $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

$1 - \tanh^2(x)$

sigmoid $f(x) = \frac{1}{1 + e^{-x}}$

$f(x)(1 - f(x))$

softmax $f(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$

$f_i(1 - f_i)$ (diagonal)

Gradient Descent:

$\theta^{k+1} = \theta^k - \eta \nabla_{\theta} J(\theta^k)$

decays over time

learning rate

$\tilde{g} = \frac{1}{B} \sum_{b=1}^B \nabla_{\theta} L(f_{\theta}(x^{(b)}), y^{(b)})$

$B \rightarrow$ batch size

$B \rightarrow$ stochastic gradient descent

With momentum: (smooths oscillations, faster convergence)

$V_{t+1} = \mu V_t + \eta \nabla_{\theta} J(\theta_t)$

$\theta_{t+1} = \theta_t + V_{t+1}$

Gradient clipping: clamp magnitude of gradient to some max.

Cross Entropy Loss Classification

Given the distribution p , predicted distribution q

$CE = -\sum p(x) \log q(x) = -\log q(x_{true})$

$\frac{dCE}{dq} = -\frac{1}{q}$

CNNs:

output size = $\left\lfloor \frac{\text{input size} + 2 \times \text{padding} - \text{dilation} \times (\text{kernel size} - 1)}{\text{stride}} \right\rfloor + 1$

characteristics of identifying images $\rightarrow$ visual hierarchy, spatial locality, translational invariance

padding, # of filters, filter size, stride

Applying filters: dot product

output dimensions: $? \times ? \times \# \text{ of filters}$

Max-pooling: size $(k \times k)$, stride, no padding

typical "pyramid" stack $\rightarrow$ image sizes get smaller

(want to find local pattern) in layers of processing

RNNs: Backprop through time

predictions

$h(t) = \sigma_1(W h_{t-1} + U x_{in}(t) + b)$

$y(t) = \sigma_2(V h_t + c)$

$\frac{\partial y(t)}{\partial x(0)} = \frac{\partial y(t)}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \dots \frac{\partial h_1}{\partial x(0)}$

Transformers:

1. tokenization: tokens $\rightarrow$ vector of neurons

2. attention mechanisms

Self-Attention ($Q, K, V \in \mathbb{R}^{n \times d_k}$)

query keys values

d_q, d_k, d_v

tokens $\rightarrow$ weighted linear combinations of other tokens

generally based on similarity score

permutation equivariant

translation equivariant

CNN: translation equivariant

can approximate any continuous function $\rightarrow$ 2-layer neural network

universal approximator

Approximation Theorem:

Suppose we use a continuous activation function that is not a polynomial. Then for any continuous function $g: \mathbb{R}^d \rightarrow \mathbb{R}$ and any error $\epsilon > 0$, there exists a 2-layer network, f , with bias terms, such that

$|f(x) - g(x)| \leq \epsilon$ for all $x \in [0, 1]^d$

use a non-linear activation function

L-Lipschitz: function $g: \mathbb{R}^d \rightarrow \mathbb{R}$, constant $L \geq 0$

if $|g(x) - g(x')| \leq L \|x - x'\|_{\infty}$

$\|x - x'\|_{\infty} = \max_{1 \leq i \leq d} |x_i - x'_i|$ (max distance)

Euclidean distance: $\|x - x'\|_2 = \sqrt{\sum_{i=1}^d (x_i - x'_i)^2}$

Theorem: $\exists$ a 3-layer ReLU network f with $\Omega((L/\epsilon)^d)$ units s.t.

$\int_{[0,1]^d} |f(x) - g(x)| dx \leq 2\epsilon$

Since g is L-Lipschitz, moving the input by Δx can change g by at most $L|\Delta x|$.

So if each bin has width $\Delta x \leq \frac{\epsilon}{L}$, then g can change by $\leq \epsilon$ within a bin.

Thus, $N = L/\epsilon$ bins can guarantee ϵ error.

MLP: if $u \leq x < v$: return w

block of height w from $x = u$ to $x = v$.

$\text{ReLU}(x) - \text{ReLU}(x-1) \} u(x, 0, 1)$

$u(x, t, g) = \text{ReLU}(x-t) - \text{ReLU}(x-t-g)$

Shift entire

Shift relative to each other

100 $\cdot u(x, 0, 0.01)$ small g

big scale

Depth Separation: Scaling depth is exponentially more efficient than scaling width.

Approximation: Best $R(\theta^*)$

Optimization: min. $\hat{R}(\theta)$

Generalization: $\hat{R}(\theta)$ vs. $R(\theta)$

Empirical Risk:

$\hat{R}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i)$

Population Risk:

$R(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{L}} \ell(f_{\theta}(x), y)$

ResNet: CNN with residual connection (connection from previous block)

ex. $X_{out} = F(X_{in}) + X_{in}$

change dimensionality: $X_{out} = F(X_{in}) + W X_{in}$

Once upon a $\rightarrow$ predictor $\rightarrow$ time

Autoregressive Models: next token predictor

CNN x_1, x_2, x_3, x_4, x_5

RNN x_1, x_2, x_3, x_4, x_5

Attention x_1, x_2, x_3, x_4, x_5

Masked Attention

Causal mask $\rightarrow$ based on previous tokens.

Translation Equivariance: output shifts when input shifts

Invariance: output stays the same when input shifts

mask later tokens, only attend on past

autoregressive $\rightarrow$ past + current

Derivative of $\log(x)$:

$\frac{d}{dx} (\log x) = \frac{1}{x \ln(a)}$

Quotient Rule:

$\frac{d}{dx} \left[\frac{f(x)}{g(x)} \right] = \frac{g(x)f'(x) - f(x)g'(x)}{(g(x))^2}$

Backward Propagation: Chain rule

$y = X_{out}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

Derivative of $\log(x)$:

$\frac{d}{dx} (\log x) = \frac{1}{x \ln(a)}$

Quotient Rule:

$\frac{d}{dx} \left[\frac{f(x)}{g(x)} \right] = \frac{g(x)f'(x) - f(x)g'(x)}{(g(x))^2}$

Backward Propagation: Chain rule

$y = X_{out}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}$

$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial X_{out}} \cdot \frac{\partial X_{out}}{\partial W}</$

GNN = graph neural networks
 ↳ like MLPs, but nodes are vectors, not scalars
 edges → complex functions → aggregate and update (like MLP) (linear) (pointwise)
Transformers: (special type of GNN)
 limitation of CNN - apart image patterns don't interact
 Token: a vector of neurons
 $T = N$ neurons, d channels = $N \times d$ $\begin{bmatrix} \text{neurons} \\ \text{tokens} \end{bmatrix}$
 $T_{out} = W T_{in} = \begin{bmatrix} F_0(T_{in,0}, \dots) \\ F_1(T_{in,0}, \dots) \\ \vdots \\ F_{L-1}(T_{in,L-1}, \dots) \end{bmatrix}$ token
 F typically MLP

Generalization:

- Model's ability to use what it has learned from training data to make accurate predictions on unseen data.
- Occam learning → Principle that among competing explanations fitting the data, simplest ones generalize well.
- Hypothesis Space → set of all possible models - larger hypothesis space ↑ flexibility, ↑ risk of overfitting
- Version Space → subset of hypothesis space containing models that fit perfectly the training data.

Countable Hypothesis Bound:

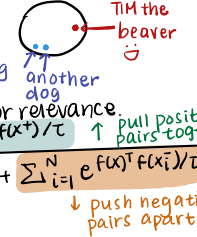
Bounded Risk $R(h, x) \in [a, a + \Delta]$
 Countable hypothesis space $h \in H$
 Needs to be independent → prior $P(h)$ of training data
 Empirical Risk: sum of $R(h, x_i)$ for fixed h independent rand. vars.

$$R(h) \leq \hat{R}(h) + \Delta \sqrt{\frac{\log(\frac{1}{\delta}) + \log(\frac{1}{\delta})}{2n}}$$

$R(h)$: population risk
 $\hat{R}(h)$: Empirical risk
 $P(h)$: prior probability of hypothesis h .
 δ : confidence parameter
 n : sample size

Idea: Hypothesis with low empirical risk + high prior probability will also have low population risk with high probability.

Inequality-universal → If prior assigns low probability to the true hypothesis - bound becomes loose but never invalid.
 Helpful priors reflect realistic assumptions about the world, such as preferring smoother, simpler, or lower-norm functions that are less likely to overfit noise.



InfoNCE loss:

Encoder maps data onto hypersphere $f: X \rightarrow S^{d-1}$
 cross-entropy loss with softmax as a "classifier" for relevance.

$$\min_f \mathbb{E}_{(x, x^+) \sim p_{pos}, \{x_i^-\}_{i=1}^N \sim p_{data}} \left[-\log \frac{e^{f(x)^T f(x^+)})}{e^{f(x)^T f(x^+)}) + \sum_{i=1}^N e^{f(x)^T f(x_i^-)}} \right]$$

Architecture - Modeling language via Decoder-only Transformers.

Pre-training: Giving LLMs broad knowledge of language, the world, and excellent foundations for reasoning and fine-tuning.

Post-training: Teaching LLMs to be instruction-following assistants and to be effective at math, coding, using tools, etc.

Tokenizing: BPE (Byte-Pair Encoding) "can't dislike this"
 Subword tokens ["can", "t", "dis", "like", "this"]
 Start with characters. Count adjacent pairs across Corpus.
 Merge most frequent pair into one new token. Repeat.

PSET 1:

1. Two-layer ReLU network: width k (k hidden neurons)
 $f(x; W_1, W_2, b_1, b_2)$ $W_1 \in \mathbb{R}^{k \times 1}$ $b_1 \in \mathbb{R}^{k \times 1}$
 l : # of weight matrices $W_2 \in \mathbb{R}^{1 \times k}$ $b_2 \in \mathbb{R}^{1 \times 1}$
 $l=3$ layers: $f(x; W_1, W_2, W_3, b_1, b_2, b_3)$
 $= W_3 \text{ReLU}(W_2 \text{ReLU}(W_1 x + b_1) + b_2) + b_3$
 slope is non-decreasing
 For ReLU network with $l \geq 2$ layers:
 → not convex } A ReLU function itself is convex
 → not concave } not concave → kink when input = 0
 negative weights break convexity
 ReLU: l layers, each of width k .
 function is continuous → each layer $w x + b$ is continuous & ReLU continuous. # of discontinuities = 0
 Output: Piecewise-linear (output is linear within regions separated by kinks)
 Smallest # of input points so that ReLU is always active: 0
 ex. $W_1 = 0, b_1 = 20$ → function constant & differentiable
 For $l=2$ layers, max k input points at which function can be non-differentiable. Since $l=2 \rightarrow 1$ ReLU & each of the k hidden neurons contributes ≤ 1 kink.

For a general number of layers l , largest # of input points at which function can be differentiable is exponential in l . → Each of the k neurons separating hyperplane of previous layer.

ReLU Logic Gates:
 OR: $f(x) = \text{ReLU}(x_1) + \text{ReLU}(x_2)$
 AND: $f(x) = \min(x_1, x_2) = \frac{1}{2}(x_1 + x_2 - |x_1 - x_2|)$
 $f(x) = \text{ReLU}(x_1 + x_2) - \text{ReLU}(-x_1 - x_2) - \text{ReLU}(x_1 - x_2) - \text{ReLU}(x_2 - x_1)$
 XOR: $f(x) = \begin{bmatrix} 1 & 1 \end{bmatrix} \text{ReLU} \left(\begin{bmatrix} -1 & 1 & 1 & -1 \end{bmatrix} \text{ReLU} \left(\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} x \right) \right)$

Augmentation: Shifts for more variants
 → improves generalizing, reduces overfitting

Attention:

↳ A: function of some input data telling us which token to attend
 query, key, value
 $q = W_q t, k = W_k t, v = W_v t$
 $s = [q_1^T k_1, \dots, q_n^T k_n]$
 $A = \text{softmax}(s)$
 $T_{out} = \begin{bmatrix} a_1 v_1^T \\ \vdots \\ a_n v_n^T \end{bmatrix}$
 Self-attention:
 all tokens
 $Q_{in} = T_{in} W_q^T, K_{in} = T_{in} W_k^T, V_{in} = T_{in} W_v^T$
 $T_{out} = A V_{in}, A = f(T_{in}) = \text{softmax} \left(\frac{Q_{in} K_{in}^T}{\sqrt{d_m}} \right)$

Pre-training Objectives:

- Supervised learning → with human annotated labels (e.g., classes)
- unsupervised learning → without human annotated labels
- self-supervised learning → labels induced by data "itself"
- modern unsupervised learning in camouflage.

Reconstruction:

- K-means clustering - assign nearest center
- Principle Component Analysis (PCA) → $\min \sum_W \|x - W^T W x\|^2$ s.t. $W^T W = I_{k \times k}$
- Autoencoder → represent data by compressing it into a low-dimensional latent space.

Idea Behind LLMs:

- don't design biases manually anymore - learn from massive diverse data.

Autoregressive Decoding Process:

1. Tokenize input → embed (KV cache)
2. "Prefill" Step: compute attention keys/values
3. Generation loop:
 - sample next token
 - append to sequence
 - update KV cache → enables fast generation

Information Theory:

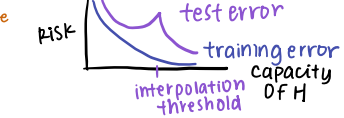
Mutual information (MI) $I(X; Y)$: how much knowing X reduces uncertainty about Y .
 • High $I(X; Z)$ → Representation Z retains info about input X .
 • High $I(Z; Y)$ → Z useful for predicting Y (task-relevant info)

Goal: keep relevant info → discard noise → Compress redundancy

Information Bottleneck perspective:

Find representation Z that:
 $\max I(Z; Y)$ s.t. $I(Z; X) \leq C$
 Compression + prediction

Double Descent: over-parameterized → better



Simple + Spiky Hypothesis:

Model spiky on training data → still do well.

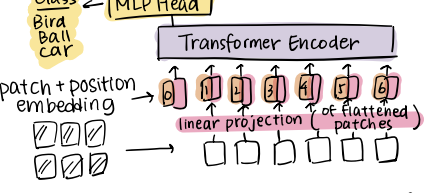
Scaling Laws:

- Model performance follows predictable trends with compute, data, & parameters
- As scale grows, self-supervised pretraining on diverse data dominates.
- Precise Engineering (parallelization, tokenization, deduplication) determines real-world performance.

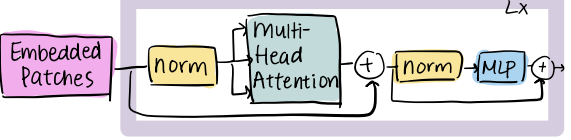
PSET 2:

CNN architecture performs better than MLPs on image data. → capturing patterns through convoluting surrounding pixels of a pixel.
 Inductive biases → translation invariance
 MLPs: treat each pixel independently (no shared parameters)
 RNN: If sequence length T very long → input has diminishing effect on output.
 OLT flops forward pass → sequential
 Self-attention: $O(L^2)$ flops → pairwise inner products & weighted sum over all timestamps
 Parallelization: transformers will be faster. T^2 operations in parallel.
 Transformers: divide continuous data into patches.
 attending on tokens that matter embeddings
 Self-attention to relate local & global relationships (+ positional encoding)

Vision Transformers (ViT):



Transformer Encoder



CNN: inductive biases → good for small datasets → flexibility for large datasets.
 ViT: learn spatial pattern without bias constraints → good for large datasets
 For different image sizes: CNN's structures are able to preserve spatial relationships & therefore generalize better than ViTs.
 ViTs rely on fixed positional encodings & so different input resolutions can disrupt learned spatial orders, unless positional encodings are adjusted relative to different resolutions.

PSET 3:

- generation speed might be slower.
- Nucleus Sampling: select tokens that have a cumulative prob. > P . more diverse results - not always token w/ highest prop.
- KV caching: → calculate attention using cached K & V for computing output.